



UCG

Univerzitet Crne Gore

OPERATIVNI SISTEMI

Shell programiranje

Promjenljive, uslovne konstrukcije, petlje i funkcije

Velibor Došljak

01

Osnovi shell programiranja



Operativni sistemi

Uvod

- Shell program (skripta) je datoteka sa nizom Linux komandi koje se izvršavaju sekvencijalno ili su povezane programskim strukturama
- Shell program se ne prevodi (ne kompajlira), već ga komandni interpreter izvršava liniju po liniju
- Ulaz se zadaje u komandnoj liniji ili interaktivno, a izlaz se može prikazati na ekranu
- Shell program se piše za određeni komandni interpreter (kod nas bash)
- Shell programiranje obuhvata
 - Tipove podataka (koristi se univerzalni tip promjenljive)
 - Naredbe (interne ili eksterne)
 - Programske strukture (petlje, uslovni skokovi, funkcije)

Prvi shell program

- Skripta se piše u bilo kom tekstualnom editoru, npr. nano ili vi (Čas 3)
- Datoteci treba dati pravo izvršavanja
- Skripta se zatim pokreće iz komandne linije

Datoteka ss1:

```
#!/bin/bash  
# prvi shell program  
clear  
echo "Hello World!"
```

Pokretanje:

```
$ chmod +x ss1  
$ ./ss1
```

Pokretanje shell programa

- Pokretanje shell programa uključuje dva koraka
 - Dodjela dozvole za izvršavanje: `$ chmod +x ss1`
 - Pokretanje programa iz komandne linije: `$ ./ss1`
- Ako se program ne nalazi u tekućem direktorijumu, potrebno je navesti putanju do programa
- Prva linija `#!/bin/bash` određuje komandni interpreter u kome će se program izvršavati
- Shell program može da se izvrši i bez prava izvršavanja, navođenjem interpretera:
 - `$ bash ss1`

02

Promjenljive



Promjenljive u Linux operativnom sistemu

- Promjenljive omogućavaju čuvanje podataka u operativnoj memoriji
- Na Linux sistemima postoje dva tipa promjenljivih:
 - Systemske promjenljive, npr. BASH, HOME, PATH, PS1, PWD, USER...
 - Korisnički definisane promjenljive
- Sve promjenljive vidljive iz tekućeg shell-a prikazuju se komandom `set`
- Vrijednost promjenljive se prikazuje sa `echo`:
 - `$ echo $PATH`

Korisničke promjenljive

- Svaka promjenljiva je univerzalna i nema deklaraciju tipa
- Promjenljiva se definiše sa ime=vrijednost, npr. `$ br=10`
 - Oko znaka = **ne smije** biti razmaka (`br = 10` je greška)
- Ime počinje slovom ili donjom crtom, a zatim slijede slova, cifre ili donje crte
 - Imena ne smiju sadržati specijalne znake (poput `?` ili `*`)
 - Imena su osjetljiva na mala i velika slova (`br` i `BR` su različite promjenljive)
- Vrijednost promjenljive `var` prikazuje se sa `$ echo $var`
- NULL promjenljiva nema vrijednost: `$ br=` ili `$ br=""`
- Promjenljiva se uklanja komandom `$ unset br`

Korisničke promjenljive – primjeri

- 1 Definirati promjenljive `x` i `xn` sa vrijednostima `10` i `abc` i prikazati ih u istom redu
- 2 Prikazati zbir dva broja na ekranu
- 3 Definirati `x=20`, `y=5` i promjenljivu `z` kao njihov količnik, pa rezultat prikazati u jednom redu

```
$ x=10; xn=abc  
$ echo $x $xn  
10 abc
```

```
$ echo $((3 + 4))  
7
```

```
$ x=20; y=5  
$ z=$((x / y))  
$ echo x/y=$z  
x/y=4
```

Komanda echo

echo prikazuje tekst ili vrijednost promjenljive na ekranu: `$ echo [opcije] [string, promjenljive...]`

- `-n` – ne prelazi u novi red nakon ispisa
- `-e` – interpretira specijalne karaktere sa kosom crtom:
 - `\n` – novi red
 - `\t` – tabulator
 - `\a` – zvučno upozorenje
 - `\\` – obrnuta kosa crta

Primjer:

```
$ echo -e "Linux\n\t\tOS !\n"
Linux
           OS !
```

Navodnici

- Dvostruki navodnici `"..."` – sve unutar njih gubi posebno značenje, osim `$`, `\` i ```
- Jednostruki navodnici `'...'` – sve unutar njih ostaje doslovno, nepromijenjeno
- Obrnuti navodnici ``...`` – izraz unutar njih se izvršava kao komanda i zamjenjuje svojim izlazom

```
$ echo "Danasnji datum je: date"
Danasnji datum je: date
$ echo "Danasnji datum je: `date`"
Danasnji datum je: Mon Oct  5 16:30:35 CEST 2026
$ echo 'Vrijednost je $HOME'
Vrijednost je $HOME
```

Zamjena komande – `$( )` umjesto obrnutih navodnika

- Obrnuti navodnici ``command`` su starija sintaksa za zamjenu komande njenim izlazom
- Savremena bash sintaksa je `$(command)` – čitljivija je i može se ugnježdavati

```
$ echo "Danasnji datum je: $(date)"  
$ echo "Broj fajlova: $(ls | wc -l)"
```

- Ugnježdavanje: `$(naredba1 $(naredba2))` – sa obrnutim navodnicima zahtijeva `\`` unutar ```, što je nepregledno

Preporuka

U novim skriptama koristiti `$( )` umjesto obrnutih navodnika

Aritmetika: `$((...))`

- Aritmetički izraz `$((izraz))` računa vrijednost cjelobrojnog izraza
- Operatori: `+`, `-`, `*`, `/` (cjelobrojno dijeljenje), `%` (ostatak)
- Unutar `$((...))` promjenljive se mogu pisati i bez `$`, a razmaci nisu obavezni
- U starijim skriptama se sreću `expr` (`z=`expr $x * $y``, sa obaveznim razmacima i `\*`) i `let` (`let zbir=5+3`)

```
$ echo $((1 + 3))  
4  
$ echo $((20 % 3))  
2  
$ i=5  
$ i=$((i + 1))  
$ echo $i  
6  
$ echo 6+3=$((6 + 3))  
6+3=9
```

Komanda read

- read čita ulazne podatke sa tastature i smješta ih u promjenljive
 - Sintaksa: `$ read var1 var2 ... varN`
- Primjer: program `ss2` čita podatak sa tastature u promjenljivu `var1` i ispisuje ga na ekran

```
#!/bin/bash
# ss2
echo "Unesite podatak: "
read var1
echo "Unijeli ste: $var1"
```

Shell program i komandni argumenti

Argumentima komandne linije se iz shell programa pristupa pomoću promjenljivih:

- `$0` – ime programa
- `$1, $2, ..., $9` – prvi, drugi, ..., deveti argument
- `$#` – broj argumenata
- `$*` – svi argumenti

Primjer:

```
#!/bin/bash
# ss3
echo "Program: $0"
echo "Broj argumenata: $# "
echo "Svi argumenti su redom: $*" 
```

```
$ ./ss3 a b c
Program: ./ss3
Broj argumenata: 3
Svi argumenti su redom: a b c
```

Izlazni status komande

- Linux komande vraćaju vrijednost na osnovu koje se može odrediti da li je komanda izvršena uspješno
- Povratna vrijednost 0 znači da je komanda uspješno izvršena, a svaka druga vrijednost predstavlja izlazni ili dijagnostički status
- Izlazni status poslednje komande čuva se u promjenljivoj \$?

```
$ rm proba  
rm: cannot remove 'proba': No such file or directory  
$ echo $?  
1
```

- Skripta postavlja svoj izlazni status naredbom `exit n`

03

Uslovne konstrukcije



Operativni sistemi

Konstrukcija if

- Tri oblika:
 - if-then-fi
 - if-then-else-fi
 - if-then-elif-else-fi
- Uslov je ispunjen ako je izlazni status 0
- Uslov može biti:
 - izlazni status neke komande
 - [izraz] ili test izraz

Najprostiji oblik:

```
if condition
then
    command1
    ...
    commandN
fi
```

Konstrukcija if – primjeri

Uslov je izlazni status komande:

```
#!/bin/bash
# ss4
if rm $1
then
    echo "Datoteka $1 je izbrisana"
fi
```

Uslov je izraz u []:

```
#!/bin/bash
# ss5: da li je argument
# pozitivan broj
if [ $1 -gt 0 ]
then
    echo "$1 je pozitivan broj."
fi
```

Pažnja

Oko [i] obavezno idu razmaci: [\$1 -gt 0], a ne [\$1 -gt 0]

Operatori poređenja

Cijeli brojevi

- `-eq` – jednako (`==`)
- `-ne` – različito (`!=`)
- `-lt` – manje od (`<`)
- `-le` – manje ili jednako (`<=`)
- `-gt` – veće od (`>`)
- `-ge` – veće ili jednako (`>=`)

Stringovi

- `str1 = str2` – stringovi su jednaki
- `str1 != str2` – stringovi su različiti
- `-n str1` – `str1` nije prazan
- `-z str1` – `str1` je prazan

Testovi nad datotekama i logički operatori

Datoteke i direktorijumi

- `-f file` – postoji i obična je datoteka
- `-d dir` – postoji i direktorijum je
- `-s file` – postoji i nije prazna
- `-r file` – ima pravo čitanja
- `-w file` – ima pravo upisa
- `-x file` – ima pravo izvršavanja

Logički operatori

- `! izraz` – negacija
- `izraz1 -a izraz2` – I (AND)
- `izraz1 -o izraz2` – II (OR)

Primjer:

```
if [ -f $1 -a -r $1 ]  
then  
    cat $1  
fi
```

if-then-else-fi

Sintaksa:

```
if condition
then
    command1-1
    ...
else
    command2-1
    ...
fi
```

Primjer: da li je argument pozitivan ili negativan broj

```
#!/bin/bash
if [ $# -eq 0 ]
then
    echo "$0: Morate navesti jedan numericki arg."
    exit 1
fi
if [ $1 -gt 0 ]
then
    echo "$1 je pozitivan."
else
    echo "$1 je negativan ili nula."
fi
```

if-then-elif-else-fi

Primjer: da li je argument pozitivan, negativan ili nula

```
#!/bin/bash
if [ $1 -gt 0 ]
then
    echo "$1 je pozitivan."
elif [ $1 -lt 0 ]
then
    echo "$1 je negativan."
elif [ $1 -eq 0 ]
then
    echo "Argument je nula."
else
    echo "$1 nije numericki argument"
fi
```

Konstrukcija case

Sintaksa:

```
case $var in
    pattern1)
        commands;;
    pattern2)
        commands;;
    *)
        commands;;
esac
```

Grana se završava sa `;;`, a `*`) odgovara svim ostalim vrijednostima

Primjer:

```
#!/bin/bash
if [ -z "$1" ]
then
    echo "Unesite korisnicko ime"
    exit 1
fi
case $1 in
    "korisnik1") echo "Korisnik1";;
    "korisnik2") echo "Korisnik2";;
    *) echo "Nepoznat korisnik!";;
esac
```

04

Petlje



Petlja while

Sintaksa:

```
while [ condition ]  
do  
    command1  
    ...  
    commandN  
done
```

Petlja **until** je obrnuta: tijelo se izvršava sve **dok uslov nije** ispunjen

Primjer: tablica množenja za dati broj

```
#!/bin/bash  
if [ $# -eq 0 ]  
then  
    echo "Argument nije naveden."  
    exit 1  
fi  
n=$1  
i=1  
while [ $i -le 10 ]  
do  
    echo "$n * $i = $((n * i))"  
    i=$((i + 1))  
done
```

Petlja for

Sintaksa:

```
for name [in words...]  
do  
    commands  
done
```

Promjenljiva name redom
dobija svaku vrijednost iz liste

Primjer: tablica množenja za dati broj

```
#!/bin/bash  
if [ $# -eq 0 ]  
then  
    echo "Argument nije naveden"  
    exit 1  
fi  
n=$1  
for i in 1 2 3 4 5 6 7 8 9 10  
do  
    echo "$n * $i = $((n * i))"  
done
```

Naredba select

Sintaksa:

```
select name [in words...]  
do  
    commands  
done
```

- Prikazuje numerisan meni i čeka izbor
- PS3 – tekst upita
- REPLY – broj koji je korisnik unio
- break – izlazak iz menija

Primjer:

```
#!/bin/bash  
PS3="Izaberite opciju: "  
select op in Pozdrav Kraj  
do  
    case $op in  
        Pozdrav) echo "Pozdrav.";;  
        Kraj) echo "OK."; break;;  
        *) echo "Opcija $REPLY ne postoji.";;  
    esac  
done
```

05

Funkcije



Funkcije u shell skriptama

- Funkcija je imenovani blok komandi koji se može pozivati više puta u skripti
- Funkcija se poziva navođenjem njenog imena, kao i svaka druga komanda
- Mora biti definisana prije prvog poziva u skripti

Sintaksa:

```
ime_funkcije () {  
    komanda1  
    ...  
    komandaN  
}
```

Primjer:

```
#!/bin/bash  
# pozdrav.sh  
pozdrav () {  
    echo "Zdravo, $1!"  
}  
pozdrav Ana  
pozdrav Marko
```

Funkcije – argumenti i povratna vrijednost

- Unutar funkcije, \$1, \$2, ... su argumenti sa kojima je **funkcija** pozvana, a ne argumenti skripte
- Naredba `return` vraća izlazni status funkcije (broj 0–255), a ne rezultat
- Rezultat se obično ispisuje sa `echo` i hvata sa `$( )`

```
#!/bin/bash
zbir () {
    echo $(( $1 + $2 ))
}
rezultat=$(zbir 3 4)
echo "Zbir je $rezultat"
```

06

Zadaci



Operativni sistemi

Zadaci

- 1 Napisati shell skriptu koja sve datoteke koje korisnik navede spaja u datoteku `sve.txt`
- 2 Napisati shell skriptu koja se poziva sa `./stamp ime`
 - ako je `ime` datoteka, prikazuje njen sadržaj
 - ako je `ime` direktorijum, lista njegov sadržaj
 - u ostalim slučajevima prikazuje poruku o grešci
- 3 Napisati skriptu koja pomoću `select` nudi izbor operacije `add`, `mul` ili `sub`, zatim učitava dva broja i na njima primjenjuje odabranu operaciju

Hvala na pažnji

Pitanja?